Departamento de Informática
Universidad Carlos III de Madrid

Bachelor in Data Science and Engineering

Machine Learning I
Course 2025-2026

Assignment 1: Classification and Prediction

Model Selection with Weka and Deployment with Scikit-learn

March 3, 2026

This assignment consists of applying Machine Learning techniques to perform classification and prediction tasks in the LunarLander-v3 environment using Weka for model analysis and comparison, and Scikit-learn for implementation and deployment. In addition, the selected model will be integrated into the environment to build an automatic agent capable of controlling the lander.

1 Introduction

In the LunarLander-v3 environment, the objective is to land a spacecraft safely on the designated landing pad while maximizing the total reward obtained during the episode. The environment provides continuous state information describing the position, velocity, angle, and contact status of the lander.

For the development of this assignment, the provided simulator will be used to control the Lunar Lander either manually (keyboard control) or using a rule-based agent. During gameplay, relevant information from the environment will be collected in order to generate datasets that will later be used to train Machine Learning models.

The goal of this assignment is to perform **behavioral cloning**, that is, to learn a policy from recorded gameplay data. Students will:

- Predict the next action of the lander given the current state (classification problem).
- Predict the immediate reward of the next frame given the current state and executed action (regression problem).

To achieve this, a set of training examples (learning experiences) must be generated and used as input for both classification and regression techniques.

More precisely, this assignment consists of:

- **State representation and feature engineering**: Design the representation of the LunarLander state by selecting relevant variables from the environment and creating additional engineered features derived from them. Students must justify the chosen attributes and evaluate the impact of the engineered features on model performance.
- **Instance collection**: Generate datasets by interacting with the LunarLander environment (using both keyboard control and the rule-based agent) and recording state-action-reward information.
- **Model selection with Weka**: Analyze and compare different Machine Learning algorithms using Weka, including feature selection and preprocessing techniques, in order to determine the most suitable model and feature subset.

- **Deployment with Scikit-learn**: Re-implement and train the selected model using Scikit-learn in Python, replicating the chosen preprocessing and feature configuration, and integrate it into the game loop to build an automatic Lunar Lander agent.
- **Prediction**: Train and evaluate at least two regression models to predict the immediate reward of the next timestep, analyzing how the chosen state representation affects prediction quality.

Students are required to perform at least two iterations of the full process (data collection, model selection, and deployment) if the behavior of the deployed agent does not match the expected performance observed during offline evaluation. It is common to obtain high accuracy results during experimentation that do not translate into good performance once the model is deployed in the environment. If such discrepancies are observed, students must revisit their data collection, feature representation, or model selection decisions. The entire iterative process must be carefully documented and critically analyzed.

2 Phase 1: Data Collection and State Representation

In this phase, students will design the representation of the LunarLander state and generate the datasets required for the learning tasks. This includes selecting and engineering relevant features from the environment observations, as well as collecting instances by interacting with the simulator. The quality of the state representation and the collected data will directly impact the performance and generalization ability of the models developed in later phases.

2.1 State Representation and Feature Engineering

Students are required to create and evaluate at least three additional engineered features derived from the original state variables.

These features may include transformations such as absolute values, combinations of variables, interaction terms, boolean indicators, or simplified representations of the state.

Examples of possible engineered features include:

- Absolute horizontal displacement: $|x\_position|$
- Absolute angle: $|angle|$
- Total velocity magnitude: $\sqrt{x\_velocity^2 + y\_velocity^2}$
- Boolean indicator of both legs in contact with the ground
- Indicator of fast descent (e.g., $y\_velocity < -0.5$)
- Interaction terms such as $angle \times angular\_velocity$

Students must:

- Clearly describe the engineered features.
- Justify why they may improve learning.
- Compare model performance using raw features versus engineered features.
- Not all engineered features will necessarily improve performance. Students must experimentally validate whether their engineered representation improves or degrades model performance.

2.2 Dataset Generation

In this part you will modify the *print_line_data()* function provided in the LunarLander template. This function must (1) generate datasets that Weka can load, and (2) store information required for the prediction task.

This extraction function will serve as the basis for the entire assignment, so it is vitally important that it is correctly implemented from the beginning. To do this, the following steps must be followed:

1. Modify the *print_line_data()* function so that it writes each generated instance **simultaneously** to two files:

- an `.arff` file (for Weka experimentation),
- and a `.csv` file (for Scikit-learn implementation).

Both files must contain **exactly the same instances**, with the **same attributes in the same order**. The ARFF file must include a correct header (relation name and attribute definitions with appropriate types), written only once before the `@data` section. The CSV file must include a header row with attribute names.

2. Each line in the file will be an instance that Weka can interpret. Each instance should contain all the information about the current state that you consider relevant, and, as the last attribute, the action executed in that state. This action will be the class we will try to predict in Phase 2.

3. You must also store information related to the next timestep in order to perform the regression task. Hence, **an instance should contain**: the **current state information**, the **executed action**, and the **immediate reward obtained in the next timestep**. **Note:** The regression task will consist of predicting the immediate reward obtained after executing an action in a given state.

4. Generate training and test data using **two different control sources**:

   - **Keyboard control**: students manually play the LunarLander environment.
   - **Rule-based agent**: use the heuristic agent implemented in Tutorial 1.

   For each control source, you should obtain approximately 80% training instances and 20% test instances. Test instances must not be copies of training ones.

   You must obtain at least the following files for each control source:

   - `training_keyboard.arff` and `training_keyboard.csv`
   - `test_keyboard.arff` and `test_keyboard.csv`
   - `training_agent.arff` and `training_agent.csv`
   - `test_agent.arff` and `test_agent.csv`

Each instance will contain information of three types:

- **Current state**: Attributes describing the current state of the LunarLander environment. These will be the input attributes of the learning algorithms. **It is the student's responsibility to select and justify the attributes used for the classification and prediction tasks.**
- **Performed action**: The action taken after evaluating the current state. This will be the output attribute of the classification algorithm (what we want to classify in Phase 2). Suggestion: use integer values to codify the possible actions.
- **Information of the next timestep**: The immediate reward obtained after executing the action. This will be the output attribute of the regression models (what you want to predict in Phase 4).

3 Phase 2: Classification with Weka Experimenter

Once you have generated the datasets in Phase 1, you will design and execute a systematic comparison of classification models using the **Weka Experimenter** interface (as done in Tutorial 3). The objective of this phase is to identify the most suitable combination of:

- Dataset (keyboard vs rule-based agent),
- Feature representation (raw vs engineered),
- Classification algorithm.

Experiment Design

You must:

1. Use the **Experimenter interface** in Weka (not only Explorer) to configure a classification experiment.
2. Include multiple datasets in the experiment:
   - Keyboard-generated dataset,
   - Rule-based agent dataset,
   - At least one dataset variant with engineered features.
3. Evaluate at least three different classification algorithms. At least one of them must not have been extensively covered in previous tutorials.
4. Use a consistent evaluation protocol (e.g., 10-fold cross-validation or predefined train/test splits).
5. Configure the Experimenter so that results are saved in ARFF format.

Results Analysis

After running the experiment:

1. Use the **Analyse** tab in Experimenter to compare algorithms.
2. Select an appropriate comparison metric (e.g., Percent Correct).
3. Perform the statistical significance test provided by Weka (default significance level 0.05).
4. Analyze:
   - Which dataset performs best.
   - Whether engineered features improve performance.
   - Whether differences between algorithms are statistically significant.

All results must be presented in tables and properly interpreted. Students must justify:

- The selected feature representation.
- The selected algorithm.
- The selected final model.

4 Phase 3: Building an Automatic Agent with Scikit-learn

In this phase, you will implement a classification model with the selected feature representation using **Scikit-learn** and integrate it into the LunarLander game loop. Before starting, make sure Scikit-learn is installed:

- `pip install scikit-learn pandas joblib`

You are not required to reproduce the exact internal implementation of Weka. Instead, you will implement a similar model using the Scikit-learn API.

For this phase, you must use the following classifier:

- `DecisionTreeClassifier`

The official documentation is available at:
`https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html`

Model Training

1. Load the corresponding CSV dataset with the selected feature representation with better results on the previous phase.
2. Split the dataset into training and test sets using: `train_test_split` (80% training, 20% test).
3. Train a `DecisionTreeClassifier` using default parameters (you may optionally tune `max_depth`).
4. Evaluate the model on the test set and report:
   - Accuracy
   - Confusion matrix
5. Save the trained model using `joblib`.

Agent Deployment

Modify the LunarLander game loop so that:

- The trained model is loaded at the start of execution.
- At each timestep, the current state is converted into the selected feature representation.
- The model predicts the action using `model.predict()`.
- The predicted action is executed in the environment.

Performance Comparison

Compare the performance of:

- The rule-based agent,
- The Machine Learning agent.

Report:

- Average total reward over multiple episodes,
- Success rate.

5 Phase 4: Prediction (Regression) with Weka Experimenter

This phase aims to explore Weka's regression algorithms. The experimentation procedure and its documentation will be equivalent to that of Phase 2, using the **Weka Experimenter** interface to run multiple experiments and compare models systematically.

Once the information has been obtained from a series of games played by both the keyboard and the rule-based agent, a prediction model must be built. To do this, follow the next steps:

1. The aim of the regression model is to predict, given the **current state** of the LunarLander and the **executed action**, the **immediate reward obtained in the next timestep**.
2. Data preprocessing: equivalent to the process described in Phase 2. You must analyze the effect of different feature subsets, engineered features, and any preprocessing technique required by the selected regressors.
3. **Important:** For this phase, the regression target (next timestep reward) must be used as the class attribute in Weka. The predicted value must not be used as an input attribute.
4. Evaluation: using the datasets generated in Phase 1, you must compare at least two different regression algorithms in Weka. The evaluation must be performed using a consistent protocol (e.g., cross-validation or predefined train/test splits), and results must be obtained through the **Experimenter** interface.
5. Compare the quality of each algorithm in an equivalent way to Phase 2. At minimum, report and compare:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)

Please, describe each experiment, represent the results, and correctly analyze them (similar to Phase 2). Results must be presented using clear tables and/or plots, and students must discuss the impact of dataset choice (keyboard vs rule-based agent) and feature engineering on prediction performance.

6 Questions

Answer the following questions:

1. What differences did you observe between training models using keyboard-generated data and rule-based agent data? Discuss possible causes for these differences in terms of state distribution and behavior variability.
2. Did engineered features improve the performance of the models? Provide a clear justification based on your experimental results and explain why some engineered features may help (or not help) certain algorithms.
3. In Phase 2, were the performance differences between algorithms statistically significant according to the Weka Experimenter analysis? What conclusions can you draw from the statistical comparison?
4. After deploying the classification model in Phase 3, did the online performance match the offline evaluation results? If not, explain possible reasons for the discrepancy.
5. If you wanted to transform the regression task (predicting the next reward) into a classification problem, what would you have to modify? Provide at least one possible discretization strategy and discuss its potential advantages and disadvantages.
6. What are the conceptual differences between predicting the next reward and directly classifying the action? Under what circumstances could reward prediction be more useful than action classification?

7 Files to Submit

All the lab assignments **must** be done in groups of 2 people. You must submit a .zip file containing the required material through Aula Global before the following deadline: **Tuesday, March 24th at 17:00**. The delivered material will be reviewed in a **face-to-face** interview on that day. The name of the zip file must contain the last 6 digits of both student's NIA, `i.e., assignment1-123456-234567.zip`

The zip file must contain the following files:

1. A **PDF** document with:
   - Cover page with the names and NIAs of both students.
   - **Phase 1:** Description of the state representation and feature engineering process. This must include:
     - Explanation of the *print_line_data()* function.
     - Description of the raw attributes and engineered features.
     - Justification of the selected representation.
     - Description of how the next-timestep reward is incorporated into each instance.
   - **Phase 2:** Description of the classification experimentation performed using the Weka Experimenter. This must include:
     - The datasets used (keyboard vs rule-based agent).
     - The evaluated algorithms.
     - The evaluation protocol.
     - Statistical comparison of results.
     - Analysis of raw vs engineered features.
     - Justification of the selected model for deployment.
   - **Phase 3:** Description of the Scikit-learn implementation and deployment of the automatic agent. This must include:

  - Explanation of how the selected model was implemented in Scikit-learn.
  - Description of preprocessing steps.
  - Offline evaluation results.
  - Online performance comparison (rule-based agent vs ML agent).
- **Phase 4:** Description of the regression experimentation performed using the Weka Experimenter. This must include:
  - The evaluated regression algorithms.
  - The evaluation metrics (MAE, RMSE, etc.).
  - Statistical comparison of results.
  - Analysis of the influence of feature engineering.
- The answers to each of the questions formulated in Section 6.
- The document must not contain screenshots of code. Weka results must not be included as screenshots; they must be presented in properly formatted tables whenever possible.
- **Conclusions.**
  - Technical conclusions about the state representation, model comparison, and deployment.
  - Discussion about differences between offline and online performance.
  - Insights about dataset quality (keyboard vs rule-based agent).
  - Description of problems encountered during the practice.
  - Personal comments and reflections.

2. The datasets generated during the assignment:
   - All ARFF files used in Weka.
   - All corresponding CSV files used in Scikit-learn.
3. The generated models:
   - The selected Scikit-learn classification model file (e.g., `.pkl`).
   - Any regression models saved from Weka experimentation.
4. The modified LunarLander source code, including:
   - The implemented *print_line_data()* function.
   - The Scikit-learn training script.
   - The integrated automatic agent.

Please, **be very careful and respect the submission rules**. The clarity of the report, the justification of the answers to the proposed questions, and the conclusions provided will be valued. The weight of this practice on the final grade for the course is 1.5 points.